

JavaScript:

Web Storage & Manipulation de Classes (classList)

Module 1: Le Stockage Web (Web Storage)

1. Introduction : Pourquoi stocker des données dans le navigateur ?

Jusqu'à présent, lorsque vous écriviez du code JavaScript et que vous actualisiez la page, toutes vos variables se réinitialisaient à zéro. C'est comme si votre application souffrait d'amnésie à chaque rechargement.

Le Web Storage est une solution moderne et intégrée aux navigateurs qui permet de sauvegarder des informations directement sur l'ordinateur de l'utilisateur sous la forme intuitive de paires Clé / Valeur.

💡 *L'analogie du bureau :*

- *Le `sessionStorage`, c'est comme un post-it posé sur votre bureau pendant votre journée de travail. Si vous quittez le bureau (fermez l'onglet), le post-it est définitivement jeté à la poubelle.*
- *Le `localStorage`, c'est comme un cahier de notes rangé dans votre tiroir. Même si vous rentrez chez vous et revenez le lendemain (fermez le navigateur ou éteignez la machine), les notes restent là.*

2. Tableau Comparatif : Local vs Session

Caractéristique	localStorage	sessionStorage
Durée de vie	Permanente (jusqu'à suppression manuelle ou script).	Temporaire (disparaît dès la fermeture de l'onglet).
Portée (Scope)	Partagé entre tous les onglets/fenêtres du même	Strictement limité à l'onglet

	site.	unique qui l'a créé.
Capacité	Environ 5 Mo à 10 Mo selon le navigateur.	Environ 5 Mo.
Type de données	Uniquement des chaînes de caractères (String).	Uniquement des chaînes de caractères (String).

3. Les 4 Commandes Magiques (La Syntaxe)

La bonne nouvelle pour l'apprentissage est que les deux objets utilisent exactement les mêmes méthodes de manipulation. Il suffit d'appeler l'un ou l'autre selon votre besoin de persistance.

A. Stocker une donnée (setItem)

```
// Syntaxe : localStorage.setItem('nom_de_la_cle', 'valeur_en_texte');
localStorage.setItem('username', 'Alice');
localStorage.setItem('theme', 'dark');
```

B. Récupérer une donnée (getItem)

```
// Syntaxe : localStorage.getItem('nom_de_la_cle');
let utilisateur = localStorage.getItem('username');
console.log(utilisateur); // Affiche à l'écran : Alice
```

C. Supprimer une donnée spécifique (removeItem)

```
localStorage.removeItem('theme'); // Supprime uniquement la clé 'theme'
```

D. Tout effacer (clear)

```
localStorage.clear(); // Vide l'intégralité du stockage lié à votre site web
```

4. Le Piège Absolu : Le stockage des Objets et Tableaux

C'est l'erreur classique commise par tout débutant. Le Web Storage ne stocke que du texte brut. Si vous tentez d'y injecter un objet ou un tableau directement, JavaScript va bêtement le convertir en texte par défaut, générant le fameux et inutile '[object Object]'.

La Solution ? Convertir l'objet en texte JSON standard à l'aller, et le re-transformer en objet natif au retour.

```
// L'objet initial à sauvegarder
const profil = { nom: "Diallo", score: 450 };

// ✗ À NE PAS FAIRE :
localStorage.setItem('user', profil);
// Résultat stocké en machine : "[object Object]" -> Totalement inexploitable !

// À FAIRE :
// 1. Convertir l'objet en chaîne de caractères JSON avant stockage
localStorage.setItem('user', JSON.stringify(profil));

// 2. Pour l'extraire et le réutiliser, on effectue l'opération inverse avec
JSON.parse()
const donneesRecuperees = localStorage.getItem('user');
const profilFormate = JSON.parse(donneesRecuperees);

console.log(profilFormate.nom); // Affiche correctement : Diallo
```

Module 2 : La Propriété classList

1. Introduction : Pourquoi préférer classList à style ?

En développement web moderne, la règle d'or est la séparation stricte des rôles : le HTML s'occupe de la structure, le CSS du design visuel, et le JavaScript uniquement du comportement dynamique.

Au lieu d'écrire des styles en ligne directement dans vos scripts (ex: `element.style.color = 'red'`), il est infiniment plus propre de déclarer des classes robustes en CSS, puis d'utiliser JavaScript pour activer ou désactiver ces classes sur vos éléments HTML.

La propriété `classList` agit précisément comme un tableau dynamique listant l'ensemble des classes d'un élément HTML.

2. Les 4 Méthodes Indispensables

Supposons que nous ayons ciblé un bouton de notre interface avec la variable suivante :

```
const monBouton = document.querySelector('button');
```

A. `.add()` : Ajouter une classe

Permet d'ajouter une ou plusieurs classes à l'élément. Si la classe est déjà présente, elle n'est pas dupliquée.

```
monBouton.classList.add('btn-actif');  
// Résultat HTML produit : <button class="btn-actif">Mon bouton</button>
```

B. .remove() : Supprimer une classe

Retire proprement une ou plusieurs classes. Si la classe visée n'existe pas, aucune erreur n'est générée par le système.

```
monBouton.classList.remove('btn-actif');
```

C. .toggle() : Le mode interrupteur

Méthode reine pour créer de l'interactivité. Si la classe est présente, elle l'enlève ; si elle est absente, elle l'ajoute automatiquement.

```
monBouton.classList.toggle('highlight');  
// Idéal pour les menus déroulants, fenêtres modales ou modes sombres.
```

D. .contains() : La vérification logique

Permet de poser une question au DOM : 'Est-ce que cet élément possède actuellement cette classe ?'. Renvoie un booléen pur (true / false), parfait pour l'intégrer dans des conditions.

```
if (monBouton.classList.contains('btn-actif')) {  
    console.log("Le bouton est actif, déclencher l'action secondaire.");  
}
```

Module 3 : Exercices d'Application Pratique

Exercice 1 : Le Mode Sombre mémorisé permanent

Objectif : Créer un commutateur de thème d'arrière-plan. Grâce au localStorage, l'ambiance choisie par l'apprenant doit persister, même si la page est actualisée ou si le navigateur est fermé.

Structure HTML & Styles CSS (index.html)

```
<!DOCTYPE html>  
<html lang="fr">  
<head>  
    <meta charset="UTF-8">  
    <title>Exercice Web Storage - ISI</title>  
    <style>
```

```

        body { transition: background 0.3s, color 0.3s; padding: 50px; font-
family: Arial, sans-serif; }
        .dark-theme { background-color: #2c3e50; color: #ecf0f1; }
        button { padding: 10px 20px; font-size: 16px; cursor: pointer; }
    </style>
</head>
<body>
    <h1>Bienvenue à l'ISI</h1>
    <p>Cliquez sur le bouton pour changer l'ambiance de la page.</p>
    <button id="theme-toggle">Passer au Mode Sombre</button>

    <script src="script.js"></script>
</body>
</html>

```

Logique JavaScript (script.js)

```

const btn = document.getElementById('theme-toggle');

// 1. Étape d'initialisation : Vérification immédiate du stockage local au
chargement
const savedTheme = localStorage.getItem('theme');

if (savedTheme === 'dark') {
    document.body.classList.add('dark-theme');
    btn.textContent = "Passer au Mode Clair";
}

// 2. Gestionnaire d'événement sur le bouton
btn.addEventListener('click', () => {
    // On bascule dynamiquement la classe CSS
    document.body.classList.toggle('dark-theme');

    // On analyse l'état final pour mettre à jour la mémoire du navigateur
    if (document.body.classList.contains('dark-theme')) {
        localStorage.setItem('theme', 'dark');
        btn.textContent = "Passer au Mode Clair";
    } else {
        localStorage.setItem('theme', 'light');
        btn.textContent = "Passer au Mode Sombre";
    }
});

```

Exercice 2 : La Boîte de Notification d'Alerte (classList)

Objectif : Manipuler plusieurs classes d'état de concert. Créer un composant graphique d'alerte contextuel que l'on peut fermer et dont on peut modifier la nature visuelle (Succès, Erreur, Info) par clic de bouton.

Code Source Structurel (index.html)

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Découverte de classList - ISI</title>
  <style>
    .notification { padding: 15px; margin: 20px 0; border: 1px solid #ccc;
border-radius: 5px; font-family: sans-serif; position: relative;}
    .close-btn { position: absolute; right: 10px; top: 10px; cursor:
pointer; font-weight: bold; }
    .success { background-color: #d4edda; color: #155724; border-color:
#c3e6cb; }
    .error { background-color: #f8d7da; color: #721c24; border-color:
#f5c6cb; }
    .info { background-color: #d1ecf1; color: #0c5460; border-color:
#bee5eb; }
    .hidden { display: none; }
  </style>
</head>
<body>
  <div id="alert-box" class="notification info">
    <span id="close" class="close-btn">X</span>
    <p id="alert-text">Ceci est une notification d'information par
défaut.</p>
  </div>
  <button id="btn-success">Mode Succès</button>
  <button id="btn-error">Mode Erreur</button>
  <button id="btn-show">Réinitialiser & Réafficher</button>
  <script src="script.js"></script>
</body>
</html>
```

Logique JavaScript Opérationnelle (script.js)

```
const alertBox = document.getElementById('alert-box');
const alertText = document.getElementById('alert-text');
const btnSuccess = document.getElementById('btn-success');
const btnError = document.getElementById('btn-error');
const btnShow = document.getElementById('btn-show');
const closeCross = document.getElementById('close');

// Fermeture de l'interface
```

```
closeCross.addEventListener('click', () => {
    alertBox.classList.add('hidden');
});

// Activation de l'état "Succès"
btnSuccess.addEventListener('click', () => {
    alertBox.classList.remove('error', 'info');
    alertBox.classList.add('success');
    alertText.textContent = "Bravo ! L'opération en base de données a réussi.";
});

// Activation de l'état "Erreur"
btnError.addEventListener('click', () => {
    alertBox.classList.remove('success', 'info');
    alertBox.classList.add('error');
    alertText.textContent = "Attention ! Une erreur de traitement réseau est
survenue.";
});

// Réinitialisation globale du composant
btnShow.addEventListener('click', () => {
    alertBox.classList.remove('hidden', 'success', 'error');
    alertBox.classList.add('info');
    alertText.textContent = "Ceci est une notification d'information par
défaut.";
});
```